

Aditya Ridho Nugroho

11231003

UAS

Link GitHub:

https://github.com/adityaridhon/Pub-Sub-Log-Aggregator_UAS-Sister

Link Demo: <https://youtu.be/epH-rid8i7k>

=====

Bagian Teori

T1 (Bab 1): Jelaskan karakteristik utama sistem terdistribusi dan trade-off yang umum pada desain Pub-Sub log aggregator.

A. Karakteristik Utama Sistem Terdistribusi

Sistem terdistribusi adalah kumpulan komputer otonom yang berkolaborasi sehingga tampil sebagai *single coherent system* (Bab 1, hlm. 2). Karakteristik utamanya meliputi:

- ***Distribution Transparency***: Menyembunyikan perbedaan arsitektur dan mekanisme komunikasi sehingga antarmuka yang diberikan konsisten dan seragam (Tanenbaum & van Steen, 2007, hlm. 2, 4).
- ***Scalability***: Relatif mudah diperluas, baik secara ukuran, sebaran geografis, maupun batasan administrasi (Tanenbaum & van Steen, 2007, hlm. 2, 9).
- ***Continuous Availability***: Sistem dirancang beroperasi terus-menerus tanpa pengguna menyadari adanya perbaikan komponen (Tanenbaum & van Steen, 2007, hlm. 2).
- ***Openness***: Mematuhi aturan standar yang memfasilitasi *interoperability* dan *portability* (Tanenbaum & van Steen, 2007, hlm. 7-8).
- ***Middleware***: Keberadaan lapisan perangkat lunak tambahan yang menyatukan jaringan komputer heterogen (Tanenbaum & van Steen, 2007, hlm. 3).

B. Trade-off Desain Pub-Sub Log Aggregator

Mengacu pada prinsip Bab 1, berikut adalah *trade-off* sistem tersebut:

- ***Distribution Transparency vs Performance***: Mengumpulkan data (log) secara asinkron antarlokasi geografis berisiko memunculkan latensi yang tinggi. Memaksakan agar latensi ini sepenuhnya tersembunyi demi transparansi justru dapat mengorbankan performa sistem secara keseluruhan (Tanenbaum & van Steen, 2007, hlm. 6-7).
- ***Replication vs Consistency***: Untuk mencegah *bottleneck* akibat server log tunggal, layanan umumnya didistribusikan dan direplikasi ke banyak *node* (Tanenbaum & van Steen, 2007, hlm. 14-15). Namun, pembaruan ke banyak replika ini memunculkan masalah konsistensi; jika sistem memaksakan *strong consistency*, mekanisme sinkronisasi globalnya justru akan membatasi skalabilitas sistem (*inherently nonscalable*) (Tanenbaum & van Steen, 2007, hlm. 15).

T2 (Bab 2): Bandingkan arsitektur client-server vs publish-subscribe untuk aggregator. Kapan memilih Pub-Sub? Berikan alasan teknis.

C. Perbandingan Arsitektur

- ***Client-Server***: Berbasis interaksi *request-reply*, di mana proses klien meminta layanan dengan mengirim pesan ke server dan menunggu balasannya (Tanenbaum & van Steen, 2007, hlm. 37). Model ini umumnya menuntut klien untuk mengetahui secara spesifik ke mana permintaan tersebut harus dikirimkan.
- ***Publish-Subscribe***: Berbasis pada perubahan *events* (Tanenbaum & van Steen, 2007, hlm. 35). Sebuah proses cukup mempublikasikan *event*, lalu *middleware* akan merutekan dan memastikan bahwa *event* tersebut hanya diterima oleh proses-proses yang telah berlangganan (*subscribe*) (Tanenbaum & van Steen, 2007, hlm. 35).

D. Kapan Memilih Pub-Sub untuk Aggregator?

Pub-Sub sangat dianjurkan untuk dipilih ketika membangun sistem pengumpul data (agregator) yang komponen-komponennya menuntut sifat *loosely coupled* atau sangat independen (Tanenbaum & van Steen, 2007, hlm. 35).

Alasan Teknis:

1. ***Decoupled in space (referentially decoupled)***: Dalam Pub-Sub, proses pengirim data (pembuat log) dan penerima (agregator) tidak perlu saling merujuk secara eksplisit (Tanenbaum & van Steen, 2007, hlm. 35). Pembuat log dapat mendelegasikan data tanpa perlu tahu alamat pasti dari aggregator yang akan mengolahnya.
2. ***Decoupled in time***: Jika Pub-Sub dikombinasikan dengan arsitektur *data-centered* menjadi *shared data spaces*, proses yang berkomunikasi tidak diwajibkan aktif secara bersamaan (Tanenbaum & van Steen, 2007, hlm. 36).

Artinya, pembuat data bisa mengirimkan log meskipun agregator sedang mati sementara, dan agregator tetap dapat mengambilnya nanti saat aktif kembali.

T3 (Bab 3): Uraikan at-least-once vs exactly-once delivery semantics. Mengapa idempotent consumer krusial di presence of retries?

- **Delivery Semantics:**
 - **At-least-once:** Menjamin pesan dipastikan sampai minimal satu kali, biasanya dengan mengandalkan mekanisme *retries* (pengiriman ulang) jika pengirim tidak menerima balasan.
 - **Exactly-once:** Menjamin pesan dikirim dan diproses tepat satu kali tanpa duplikasi, terlepas dari adanya kegagalan jaringan. Sebagai perbandingan di dalam buku, sistem seperti DCE RPC menggunakan **at-most-once** sebagai semantik *default*, di mana suatu panggilan dipastikan tidak pernah dieksekusi lebih dari satu kali meskipun terjadi *crash* (Tanenbaum & van Steen, 2007, hlm. 140).
- **Krusialnya Idempotent Consumer pada Retries:** Dalam interaksi *client-server*, saat klien tidak menerima *reply*, klien sering kali melakukan *retries* dengan mengirim ulang *request* (Tanenbaum & van Steen, 2007, hlm. 37). Masalahnya, klien tidak dapat mendeteksi apakah *request* aslinya yang hilang atau justru pesan *reply* dari server yang gagal terkirim (Tanenbaum & van Steen, 2007, hlm. 37). Jika *reply* yang hilang, *retries* akan menyebabkan server mengeksekusi operasi yang sama dua kali (Tanenbaum & van Steen, 2007, hlm. 37).
- Di sinilah *consumer* harus bersifat **idempotent**, yang berarti suatu operasi dapat dieksekusi berulang kali tanpa memberikan dampak buruk atau mengubah *state* akhir sistem (*repeated multiple times without harm*) (Tanenbaum & van Steen, 2007, hlm. 37, hlm. 140). Adanya *idempotent consumer* memungkinkan mekanisme *retries* dilakukan secara aman untuk mengatasi pesan hilang tanpa risiko merusak konsistensi data.

T4 (Bab 4): Rancang skema penamaan untuk topic dan event_id (unik, collision-resistant). Jelaskan dampaknya terhadap dedup.

Berdasarkan prinsip sistem terdistribusi, *event_id* harus dirancang sebagai **true identifier**, yaitu nama *flat* (tidak terstruktur) yang sepenuhnya unik, menunjuk pada satu entitas spesifik, dan tidak boleh digunakan ulang untuk entitas lain (Tanenbaum & van Steen, 2007, hlm. 181). Di sisi lain, skema *topic* sebaiknya dirancang menggunakan **structured naming** (seperti hierarki nama pada direktori atau DNS) agar pengelompokan entitas mudah dipahami dan ditelusuri (Tanenbaum & van Steen, 2007, hlm. 196). Dalam ekosistem *publish/subscribe*, sebuah *message broker* akan menerima

pesan dari *publisher* dan secara otomatis merutekannya ke *subscriber* berdasarkan topic yang tepat (Tanenbaum & van Steen, 2007, hlm. 151).

Rancangan ini berdampak langsung pada keandalan **deduplication (dedup)**. Pada *Message-Oriented Middleware* (MOM) yang persisten, sistem menyimpan pesan dalam antrian secara jangka menengah meskipun penerima sedang pasif (Tanenbaum & van Steen, 2007, hlm. 145). Jika terjadi jeda jaringan, klien bisa menganggap pesan hilang dan mengirimkan **retries** (Tanenbaum & van Steen, 2007, hlm. 37). Penggunaan *event_id* sebagai *true identifier* yang kebal bentrokan (*collision-resistant*) memungkinkan lapisan penyimpanan log untuk membandingkan ID secara absolut (Tanenbaum & van Steen, 2007, hlm. 181) dan membuang duplikat log secara akurat sebelum diproses lebih lanjut.

T5 (Bab 5): Bahas ordering: kapan total ordering tidak diperlukan? Usulkan pendekatan praktis (mis. event timestamp + monotonic counter) dan batasannya.

Penerapan **total ordering**, di mana semua proses di seluruh sistem menyepakati urutan kejadian yang sama persis (Tanenbaum & van Steen, 2007, hlm. 248), **tidak diperlukan** apabila penulisan data bersifat *concurrent* (berjalan bersamaan tanpa relasi kausalitas), sesuai dengan aturan **causal consistency** (Tanenbaum & van Steen, 2007, hlm. 285). Pada konsistensi kausal, eksekusi pembaruan log yang murni *concurrent* diperbolehkan terlihat dalam urutan berbeda di mesin penerima yang berbeda (Tanenbaum & van Steen, 2007, hlm. 285).

Sebagai pendekatan praktis untuk ordering aggregator, kita bisa menggunakan konsep jam logis Lamport. Untuk memastikan tidak ada *timestamp* yang identik, waktu logis (*event timestamp*) digabungkan dengan nomor identifikasi proses di bagian akhir (*monotonic counter/process number*) menjadi format seperti 40.i (Tanenbaum & van Steen, 2007, hlm. 247).

Batasannya, *logical clocks* sederhana seperti ini hanya menangkap urutan linear tetapi tidak bisa menyimpulkan apakah satu pesan *benar-benar* memengaruhi pesan lain (*causal dependency*) hanya dengan melihat nilai $C(a) < C(b)$ (Tanenbaum & van Steen, 2007, hlm. 249). Untuk melacak dependensi data secara akurat tanpa *total ordering*, sistem harus beralih menggunakan algoritma **vector clocks** yang lebih kompleks (Tanenbaum & van Steen, 2007, hlm. 249-250).

T6 (Bab 6): Identifikasi failure modes (duplikasi, out-of-order, crash). Jelaskan strategi mitigasi (retry, backoff, durable dedup store, crash recovery).

Dalam arsitektur terdistribusi seperti *log aggregator*, terdapat beberapa *failure modes* utama:

1. **Duplikasi:** Saat klien atau pengirim log mengalami batas tunggu (*timeout*), mereka tidak bisa mendeteksi apakah pesan aslinya atau balasan (*reply*) yang hilang; akibatnya klien memicu **retries** yang menciptakan duplikasi (Tanenbaum & van Steen, 2007, hlm. 37).
2. **Out-of-order:** Pesan bisa tiba tidak sesuai urutan akibat perbedaan rute di *underlying network*.
3. **Crash:** *Node* pengirim, penerima, atau koordinator jaringan mati tiba-tiba dan meninggalkan status yang ambigu (Tanenbaum & van Steen, 2007, hlm. 254).

Strategi Mitigasi:

- **Retry & Backoff:** Pengirim log selalu melakukan pengiriman ulang (*retries*) untuk menutupi kehilangan paket jaringan (Tanenbaum & van Steen, 2007, hlm. 37). (Pendekatan *backoff* eksponensial umumnya digunakan untuk mengatur jeda *retry* agar jaringan tidak tersumbat).
- **Durable dedup store:** Menggunakan *Message-Oriented Middleware* persisten yang menawarkan penyimpanan *intermediate-term* pada antrean pesan, sehingga pesan log ditampung aman walau *receiver* mati (Tanenbaum & van Steen, 2007, hlm. 145). Sistem memadukannya dengan ID unik untuk menolak duplikasi (*dedup*).
- Mitigasi **Out-of-order** dikontrol ketat oleh **vector clocks**. Jika pesan datang mendahului syarat kausalitasnya, *middleware* penerima menunda penyalurannya (*delay delivery*) hingga seluruh pesan pendahulunya tiba lengkap di antrean (Tanenbaum & van Steen, 2007, hlm. 250).

T7 (Bab 7): Definisikan *eventual consistency* pada aggregator; jelaskan bagaimana idempotency + dedup membantu mencapai konsistensi.

Pada agregator log berskala besar, **eventual consistency** adalah model jaminan bahwa apabila sistem sedang tidak menerima *update* untuk waktu yang relatif lama, seluruh replika data pada akhirnya akan terkonvergensi pelan-pelan menuju salinan yang sama persis (Tanenbaum & van Steen, 2007, hlm. 289). Model ini menoleransi latensi jaringan atau kondisi ketidakkonsistenan *sementara* dan sangat ideal jika *write-write conflicts* jarang terjadi pada basis data replikanya (Tanenbaum & van Steen, 2007, hlm. 289).

Mekanisme **idempotency dan dedup** menjadi tulang punggung untuk mencapai konsistensi ini tanpa sinkronisasi global. Karena replika menyelaraskan data secara longgar, pengirim bisa saja tanpa sengaja mengirim *log update* dua kali (*retries*). Sifat

konsumer yang **idempotent** menjamin bahwa mengulang operasi beberapa kali tetap akan menghasilkan *state* akhir yang valid dan aman (*repeated multiple times without harm*) (Tanenbaum & van Steen, 2007, hlm. 37). Dipadukan dengan *dedup* via *event_id*, *middleware* bisa secara buta menelan atau menyingkirkan pesan berulang tanpa takut merusak perhitungan akhir agregator log, memastikan replika mencapai *eventual consistency* yang valid.

T8 (Bab 8): Desain transaksi: ACID, isolation level, dan strategi menghindari lost-update.

A. Desain Transaksi: ACID

Transaksi dirancang untuk menjamin bahwa seluruh objek yang dikelola server tetap berada dalam keadaan konsisten, baik ketika diakses secara bersamaan oleh banyak transaksi maupun saat terjadi kegagalan server. Keempat sifat fundamental transaksi dirangkum dalam akronim ACID.

1. Atomicity (Atomisitas)

Transaksi harus diselesaikan secara utuh atau tidak sama sekali. Sifat ini mencakup dua aspek penting:

- Failure Atomicity: Efek transaksi tetap bersifat atomik meskipun server mengalami kegagalan (crash).
- Rollback Atomicity: Jika transaksi gagal atau dibatalkan, seluruh efeknya harus dihapus sepenuhnya tanpa meninggalkan jejak apa pun. (Coulouris et al., 2012, hlm. 680).

2. Consistency (Konsistensi)

Transaksi membawa sistem dari satu keadaan konsisten ke keadaan konsisten lainnya. Dalam sistem perbankan, total saldo seluruh akun cabang harus tetap seimbang setelah operasi transfer antara dua akun selesai dilakukan. Artinya, jumlah uang sebelum dan sesudah transfer harus sama. (Coulouris et al., 2012, hlm. 681)

3. Isolation (Isolasi)

Setiap transaksi harus berjalan tanpa gangguan dari transaksi lain yang sedang berlangsung secara bersamaan. Efek sementara (menengah) dari sebuah transaksi tidak boleh terlihat oleh transaksi lain sebelum transaksi tersebut berhasil diselesaikan (commit). (Coulouris et al., 2012, hlm. 681)

4. Durability (Daya Tahan)

Setelah transaksi berhasil diselesaikan (commit), seluruh efek perubahannya harus disimpan dalam penyimpanan permanen. Data yang telah tersimpan ini harus bertahan dan tidak hilang meskipun server mengalami kegagalan (crash). (Coulouris et al., 2012, hlm. 681)

B. Isolation Level

Server yang mendukung transaksi harus menyinkronkan operasi sedemikian rupa agar memenuhi persyaratan isolasi.

1. Serial Equivalence (Ekuivalensi Serial)

Untuk memaksimalkan konkurensi dan kinerja sistem, transaksi diizinkan dieksekusi secara bersamaan (concurrent), dengan syarat efek gabungannya sama persis dengan eksekusi serial (dijalankan satu per satu). Eksekusi semacam ini dikenal sebagai serially equivalent atau serializable. (Coulouris et al., 2012, hlm. 681)

2. Strict Executions (Eksekusi Ketat)

Untuk memberlakukan properti isolasi yang ketat dan menghindari masalah seperti dirty reads (membaca data yang belum di-commit) dan premature writes (penulisan data sebelum waktunya), sistem menerapkan eksekusi transaksi yang ketat. Dalam metode ini:

- Operasi baca (read) maupun tulis (write) akan ditunda
- Penundaan berlangsung hingga transaksi sebelumnya yang memodifikasi objek tersebut dipastikan telah di-commit atau dibatalkan (Coulouris et al., 2012, hlm. 690)

C. Strategi Menghindari Lost-Update

1. Definisi

Lost-Update terjadi ketika dua transaksi membaca nilai lama yang sama dari suatu variabel, menggunakannya untuk menghitung nilai baru, dan kemudian saling menimpa. Akibatnya, pembaruan dari transaksi pertama hilang karena ditimpa oleh transaksi kedua yang tidak melihat pembaruan pertama tersebut. (Coulouris et al., 2012, hlm. 683)

2. Strategi Utama: Serial Equivalence

Cara paling mendasar untuk menghindari lost-update adalah dengan menerapkan kriteria Serial Equivalence. Jika transaksi dieksekusi secara ekuivalen serial, masalah ini tidak mungkin terjadi karena:

- Satu transaksi akan diselesaikan terlebih dahulu sebelum transaksi lain dijalankan
- Transaksi yang datang belakangan akan selalu membaca nilai terbaru yang sudah ditulis oleh transaksi awal (Coulouris et al., 2012, hlm. 685)

T9 (Bab 9): Kontrol konkurensi: locking/unique constraints/upsert; idempotent write pattern.

A. Kontrol Konkurensi: Locking (Penguncian)

Dalam sistem terdistribusi, setiap server bertanggung jawab untuk mengontrol konkurensi objeknya secara lokal, namun sekumpulan server harus bekerja

sama untuk memastikan bahwa transaksi dieksekusi secara ekuivalen serial di tingkat global (Coulouris et al., 2012, hlm. 740).

Penerapan locking pada transaksi terdistribusi memiliki beberapa aturan dasar:

- Penguncian Lokal: Kunci (locks) pada suatu objek ditahan secara lokal di server yang sama tempat objek itu berada. Manajer kunci lokal memiliki wewenang untuk memberikan kunci atau membuat transaksi yang meminta menjadi menunggu (Coulouris et al., 2012, hlm. 740).
- Penahanan Kunci Selama Fase Commit: Kunci tidak dapat dilepaskan oleh manajer lokal sampai dipastikan bahwa transaksi tersebut telah di-commit atau dibatalkan (aborted) di semua server yang terlibat. Selama proses protokol persetujuan (seperti two-phase commit), objek akan terus terkunci dan tidak tersedia bagi transaksi lain (Coulouris et al., 2012, hlm. 740-741).
- Risiko Distributed Deadlock: Karena manajer kunci di server yang berbeda menetapkan kunci secara independen, sangat mungkin server yang berbeda memaksakan urutan transaksi yang berbeda. Hal ini dapat memicu siklus ketergantungan antar-transaksi yang dikenal sebagai distributed deadlocks (kebuntuan terdistribusi) (Coulouris et al., 2012, hlm. 741).

B. Unique Constraints dan Upsert

Dalam kontrol konkurensi terdistribusi, konsep ketunggalan (uniqueness) sangat vital untuk menyelesaikan konflik:

- Globally Unique Timestamps: Pada metode kontrol konkurensi timestamp ordering, koordinator transaksi harus menerbitkan stempel waktu yang unik secara global. Untuk menghindari duplikasi atau konflik, stempel waktu unik ini dibuat dengan menggabungkan waktu lokal dan pengenalan server (<local timestamp, server-id>). Urutan eksekusi disepakati secara global berdasarkan perbandingan pasangan pengenalan unik ini (Coulouris et al., 2012, hlm. 741).
- Transaction Identifiers (TID) yang Unik: Setiap transaksi membutuhkan pengenalan yang dipastikan unik di seluruh sistem terdistribusi. Cara sederhananya adalah TID memiliki dua bagian: pengenalan unik server pembuatnya (seperti alamat IP) dan sebuah angka lokal yang unik (Coulouris et al., 2012, hlm. 730). Pengenalan unik ini krusial agar sinkronisasi dan komitmen bisa terlacak tanpa keliru.

C. Idempotent Write Pattern

- Pentingnya Idempotensi: Sangat penting agar prosedur pemulihan transaksi bersifat idempoten, yang berarti operasi tersebut dapat dilakukan berkali-kali tanpa batas namun tetap menghasilkan efek akhir yang sama persis (Coulouris et al., 2012, hlm. 755).
- Alasan Penggunaan: Pola idempoten mutlak diperlukan karena server bisa saja kembali mengalami kegagalan (mati/ crash) ketika proses prosedur

- pemulihan sedang berlangsung. Jika pemulihan tidak idempoten, crash ganda akan merusak konsistensi objek (Coulouris et al., 2012, hlm. 755).
- Tantangan Implementasi: Pencapaian pola pemulihan idempoten ini tergolong lugas jika semua objek selalu dikembalikan ke memori volatil. Namun, untuk basis data yang menyimpan objek di penyimpanan permanen (disk) dan memiliki cache di memori volatil, mencapai operasi pemulihan/penulisan yang idempoten menjadi sedikit lebih rumit (Coulouris et al., 2012, hlm. 755).

T10 (Bab 10–13): Orkestrasi Compose, keamanan jaringan lokal, persistensi (volume), observability.

A. Orkestrasi Compose

Orkestrasi Compose adalah alat/konsep yang digunakan untuk mendefinisikan, mengkonfigurasi, dan menjalankan aplikasi multi-kontainer secara terkoordinasi. Alih-alih menjalankan perintah sistem satu per satu, pengembang menggunakan satu file deklaratif (misalnya YAML) untuk mengatur semua layanan, jaringan, dan kebutuhan penyimpanan (volume) agar aplikasi dapat beroperasi di lingkungan terisolasi dengan cara yang dapat diulang secara konsisten.

B. Keamanan Jaringan Lokal

Dalam sistem terdistribusi berbasis kontainer, keamanan jaringan lokal berfokus pada isolasi lalu lintas antar layanan (mikroservis). Hal ini memastikan bahwa sistem menerapkan prinsip *least privilege*, di mana hanya layanan yang diizinkan secara eksplisit yang dapat saling berkomunikasi melalui jaringan internal tertutup, sehingga meminimalisir risiko jika terjadi kebocoran keamanan pada salah satu titik.

C. Persistensi (Volume)

- Agar sebuah transaksi memenuhi syarat atomik, efek dari transaksi yang disetujui (di-*commit*) harus memiliki sifat durabilitas.
- Durabilitas mewajibkan agar objek-objek data disimpan di dalam penyimpanan permanen (*permanent storage*).
- Persistensi ini menjamin bahwa data akan tersedia tanpa batas waktu di masa depan dan semua efek perubahan akan aman meskipun server atau sistem mengalami kegagalan/mati secara tiba-tiba. (Coulouris et al., 2012, hlm. 751)

D. Observability

Observability di masa kini berfokus pada pelacakan metrik, *trace*, dan log untuk memahami *state* aplikasi. Dalam teori di dokumen terlampir, konsep pemantauan ini diimplementasikan dengan kuat melalui **Logging** untuk *recovery*.

- Server menggunakan teknik *logging* di mana sistem memelihara *recovery file* yang bertindak sebagai catatan riwayat historis seluruh transaksi yang dieksekusi oleh server.
- Log ini memberikan visibilitas dengan mencatat nilai objek, status transaksi (*prepared*, *committed*, atau *aborted*), serta *intentions list* secara berurutan.
- Kemampuan untuk mengobservasi riwayat ini bersifat krusial: jika terjadi *crash*, *recovery manager* dapat membaca log ini untuk merekonstruksi dan memastikan status terakhir dari transaksi mana yang harus diselesaikan atau dibatalkan. (Coulouris et al., 2012, hlm. 753)

Daftar Pustaka

Tanenbaum, A. S., & van Steen, M. (2007). *Distributed systems: Principles and paradigms* (2nd ed.). Prentice Hall.

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5th ed.). Pearson Education.